

NalandaGPT - Enterprise Software Requirements Specification (SRS) & System Design

Executive Summary

NalandaGPT is a production-grade AI platform for Nalanda Mahavihara combining conversational AI, RAG, citations, multilingual search, voice assistance, image understanding, knowledge graphs, and guided heritage tours.

Project Scope

Support researchers, students, tourists, and administrators with reliable source-backed knowledge.

Business Objectives

Preserve heritage knowledge, improve accessibility, provide AI-assisted research, and offer interactive visitor experiences.

User Roles

Guest, Registered User, Researcher, Content Manager, Administrator, Super Admin.

User Stories

Users can ask questions, upload documents, identify artifacts, continue chats, use voice queries, and explore relationships through knowledge graphs.

Functional Requirements

1. Authentication
2. Chat System
3. Knowledge Base Search
4. Hybrid Web Search
5. Citations
6. PDF Processing
7. OCR
8. Image Understanding

- 9. Voice Assistant
- 10. Knowledge Graph
- 11. Guided Tours
- 12. Admin Dashboard

Non-Functional Requirements

Performance, scalability, maintainability, reliability, security, accessibility, multilingual support.

Frontend Architecture

Next.js, React.js, JavaScript, Tailwind CSS, shadcn/ui, Zustand, TanStack Query, Cytoscape.js, Leaflet.

Backend Architecture

FastAPI with modular service-oriented architecture.

Database Design

PostgreSQL Tables:

users
roles
permissions
chat_sessions
messages
documents
chunks
citations
sources
uploads
audit_logs
tour_routes
knowledge_entities

Vector Search Design

Qdrant stores embeddings generated using BGE-M3.

Knowledge Graph Design

Neo4j stores scholars, kings, monasteries, temples, artifacts, and relationships.

Authentication Design

JWT + Google OAuth.

Document Processing Pipeline

Upload

→ OCR

→ Chunking

→ Metadata Extraction

→ Embedding Generation

→ Qdrant Indexing

Conversation System

ChatGPT-style sessions, memory, rename/delete/export chats.

Hybrid Search Architecture

Query

→ Qdrant

→ Confidence Check

→ Tavily Search

→ Source Ranking

→ Gemini

→ Citations

Citation System

Every answer includes source title, URL, confidence score, and clickable references.

Image Understanding

Artifact identification, ruin explanation, inscription OCR, visual Q&A.

Voice Assistant

Faster Whisper for STT and Coqui TTS for TTS.

Multilingual Support

English and Hindi support using BGE-M3 multilingual retrieval.

Guided Heritage Tours

AI-generated routes with maps and historical explanations.

Admin Dashboard

Document uploads, source approval, user management, re-indexing.

Security Architecture

HTTPS

JWT

RBAC

Rate Limiting

Audit Logging

Secure File Uploads

Secrets Management

API Specifications

POST /auth/login

POST /auth/register

GET /chat/sessions

POST /chat/message

POST /documents/upload

POST /search/query

POST /vision/analyze

POST /voice/transcribe

GET /graph/entities

Deployment Architecture

Docker Compose:

frontend

backend

postgres

qdrant

neo4j

redis

minio

Infrastructure Requirements

16 GB RAM, 8 vCPU, 200 GB SSD recommended.

Client-Owned Resources

Gemini API Key

Tavily API Key

Domain Name

VPS/Cloud Account

Google OAuth Credentials

Cost Optimization

Self-hosted databases, local embeddings, MinIO storage.

Backup & Recovery

Daily database backups, weekly storage backups, monthly snapshots.

Testing Strategy

Unit testing, integration testing, API testing, load testing, security testing.

Acceptance Criteria

Source-backed answers, working citations, multilingual support, image analysis, voice support, web search fallback.

Project Deliverables

Source Code
Docker Compose
Deployment Guide
API Documentation
Database Schema
Admin Manual
User Manual

Technology Stack Summary

Frontend: Next.js, React.js
Backend: FastAPI
Database: PostgreSQL
Vector DB: Qdrant
Graph DB: Neo4j
Cache: Redis
Storage: MinIO
LLM: Gemini 2.5 Flash
Embeddings: BGE-M3
Vision: Gemini Vision
OCR: EasyOCR
STT: Faster Whisper
TTS: Coqui TTS
Search: Tavily